

# Attention Is All You Need

*The Transformer architecture*

---

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin  
*Google Brain · June 2017 · NeurIPS 2017*

## WHAT YOU'LL LEARN

- Why RNN/LSTM seq2seq hit a wall around 2017
- Self-attention as the central mechanism and what Q, K, V really mean
- The full encoder-decoder architecture, sub-layer by sub-layer
- Positional encoding, multi-head attention, residuals, layer norm
- How modern LLMs (GPT, Claude, Llama) descend from this paper
- Code walkthrough, limitations, and the variants that followed

# 1. Overview

## What this paper is, in one paragraph

*Attention Is All You Need* introduced the Transformer — a neural network architecture for processing sequences that uses self-attention as its only mechanism for combining information across positions. Published by Vaswani et al. at Google Brain in June 2017, it was originally a translation paper, improving English-German machine translation by about 2 BLEU points over the previous best models. But its real importance only became clear later: every major language model since 2018 — BERT, GPT-1 through GPT-5, Claude, Llama, Gemini, Mistral — is a Transformer or a direct descendant. The paper is the single most important architectural contribution to deep learning in the last decade.

## Why it matters

Before this paper, sequence models were dominated by RNNs and LSTMs. These processed sequences *step by step*: to handle a 100-token sentence, the model had to do 100 sequential matrix operations, each waiting for the previous one to finish. This made them slow to train and hard to scale.

The Transformer changed the basic operation. Instead of stepping through the sequence, it processes all tokens in parallel through self-attention layers. What used to be 100 sequential operations becomes one parallel operation across 100 tokens. On modern hardware (GPUs, TPUs), this is dramatically faster — 3-10× in wall-clock time for the same training budget.

More importantly, the parallelism made *scale* possible. You can't train a model with hundreds of billions of parameters using sequential RNN updates — it would take forever. The Transformer is the architecture that unlocked GPT-3, GPT-4, and everything since.

## How to read this guide

This document is organized like a tutorial. Section 2 explains the problem this paper was solving — why RNNs alone weren't enough. Section 3 explains self-attention conceptually. Sections 4-6 walk through the architecture in detail, building up the full transformer one piece at a time. Section 7 covers the math (Q/K/V projections, scaled dot-product, multi-head). Section 8 walks through code. Section 9 covers limitations and what came after. Section 10 covers real-world adoption. Section 11 gives worked examples. Section 12 contains interview-style Q&A.;

If you're new to this material, read in order. If you already understand transformers and just want a refresher, skim sections 1-3 and focus on sections 5-7 and 11.

## Prerequisites

Helpful: basic linear algebra (matrix multiplication, dot products), the idea of a neural network with weights and activations, what backpropagation is at a conceptual level. Familiarity with RNNs/LSTMs helps but isn't required — we explain what they did and why they fell short.

## 2. The Problem Before Transformers

To appreciate what the Transformer changed, you need to understand what came before it. From roughly 2014 to 2017, sequence modeling — machine translation, speech recognition, summarization — was dominated by Recurrent Neural Networks (RNNs) and their better-known variant, LSTMs.

### 2.1 How RNNs processed text

An RNN reads a sequence one token at a time. At each step, it takes the current input token and the previous "hidden state" (a vector summarizing everything it has seen so far), and produces a new hidden state. For a sentence like *"The cat sat on the mat"*, the RNN would do six sequential updates — step 1 processes "The", step 2 processes "cat" while remembering "The", step 3 processes "sat" while remembering "The cat", and so on.

This worked. By 2016, neural translation systems based on LSTMs had largely replaced the older phrase-based statistical machine translation in Google Translate and other major systems. Quality improvements were real and visible to users.

### 2.2 Three problems with the RNN approach

#### Problem 1 — No parallelism across time

Step 2 can't start until step 1 finishes. Step 3 can't start until step 2 finishes. The hidden state from one step is the input to the next. This is inherently sequential, no matter how much hardware you have.

On a GPU with thousands of cores, this is brutal. A GPU wants to do many things in parallel. An RNN forces it to do them one at a time. As GPUs got faster, RNN training scaled poorly — you could buy a better GPU and not see much speedup.

#### Problem 2 — The information bottleneck

Whatever the hidden state contains is the only information passed from one step to the next. For long sequences — say, a 50-word sentence with a subject at the start and a verb at the end — the model has to compress everything important about the first 49 words into one vector and hope nothing essential gets lost.

Even LSTMs, which were designed specifically to mitigate this, hit a wall around 50-100 token sequences. They could remember things across moderate distances, not across paragraphs.

#### Problem 3 — Vanishing gradients on long sequences

Even with the LSTM's gating mechanism, training signals had to propagate backward through every step. Gradients tended to shrink (or occasionally explode) as they traveled across many time steps. The model couldn't reliably learn dependencies that spanned long sequences.

### 2.3 Attention as a partial solution (Bahdanau, 2014)

Three years earlier, Bahdanau et al. had introduced an idea called **attention** as a fix for the bottleneck problem. The idea: when the decoder generates each output word, instead of relying only on one summary

vector from the encoder, let it "look back" at *all* encoder hidden states and decide which ones are most relevant.

This was bolted onto existing LSTM seq2seq systems. It worked beautifully for translation — long sentences started to translate well again. By 2016, attention was standard in most state-of-the-art NMT systems.

But the RNNs were still there. Attention helped the decoder access information, but the underlying encoder and decoder were still LSTMs processing tokens sequentially. The parallelism problem remained.

#### THE QUESTION VASWANI ET AL. ASKED

If attention is doing all the useful work for combining information across the sequence — if it lets the decoder access every encoder state directly — do we even need the recurrent layers? What if we just used attention for everything?

## 3. The Key Idea: Self-Attention

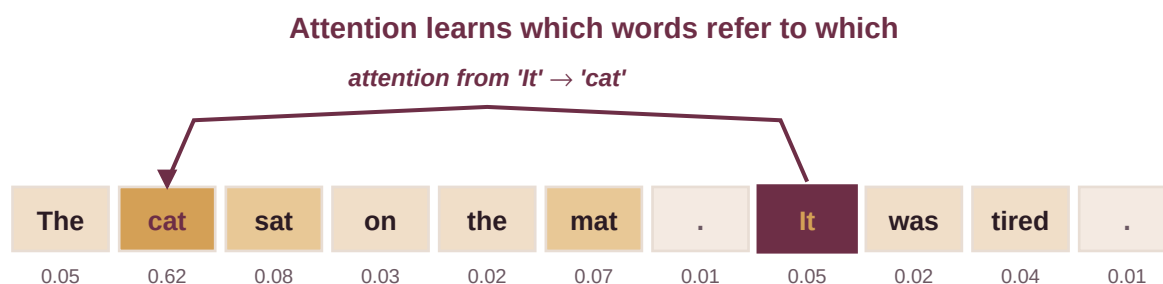
The Transformer's central operation is **self-attention**. Before unpacking the math, let's build intuition for what it actually does.

### 3.1 Attention as 'look at the relevant words'

Consider the sentence: "The cat sat on the mat. It was tired."

When you read "It" in the second sentence, you instantly know it refers to the cat. Your brain weighed every prior word and decided "cat" was the most relevant antecedent. You weren't told what "It" refers to — you inferred it from context.

Self-attention does the same thing. For each word in a sentence, the model computes a relevance score against every other word and produces a new representation that's a weighted combination of all the other words' representations — weighted by how relevant they are.



When processing "It", the model weighs every other word for relevance.

*It learned that "It" most likely refers to "cat". No rule was hardcoded — just data.*

*(The numbers below each word are attention weights. They sum to 1.0.)*

Figure 3.1 — An attention pattern from "It" to the other words in the sentence. The model has learned that "It" most strongly attends to "cat". The intensity of each box represents the attention weight. Nothing was hardcoded — these weights were learned from data.

### 3.2 Why this is so powerful

Three properties make self-attention transformative:

- 1. Direct connections across any distance.** When the model processes "It" in the example above, it can look directly at "cat" — even if they're separated by many words. There's no degradation with distance, no information bottleneck. Word 1 and word 100 are exactly one operation apart.
- 2. Fully parallel.** The attention scores between any two positions can be computed simultaneously. The whole attention matrix — every word's relevance to every other word — is one big matrix multiplication. On a GPU, this is exactly the kind of operation hardware is designed for.
- 3. Content-dependent routing.** The attention weights are computed from the words themselves. When "It" appears in a sentence about a cat, it attends to the cat. When "it" appears in "It is raining", the same model

learns to attend to different things — dynamically, based on context. There's no fixed routing, no hardcoded grammar rules.

### RNN: sequential steps vs Transformer: parallel processing

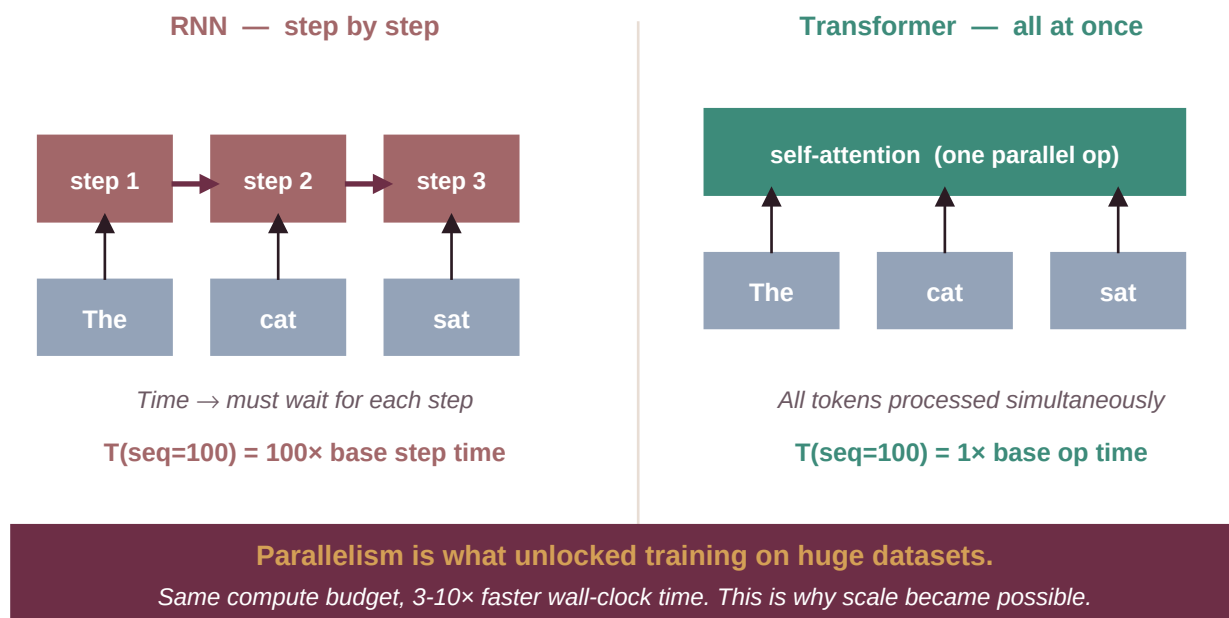


Figure 3.2 — The fundamental architectural shift. RNNs forced sequential processing, which scaled poorly on parallel hardware. Transformers process the entire sequence at once, which scales beautifully on GPUs.

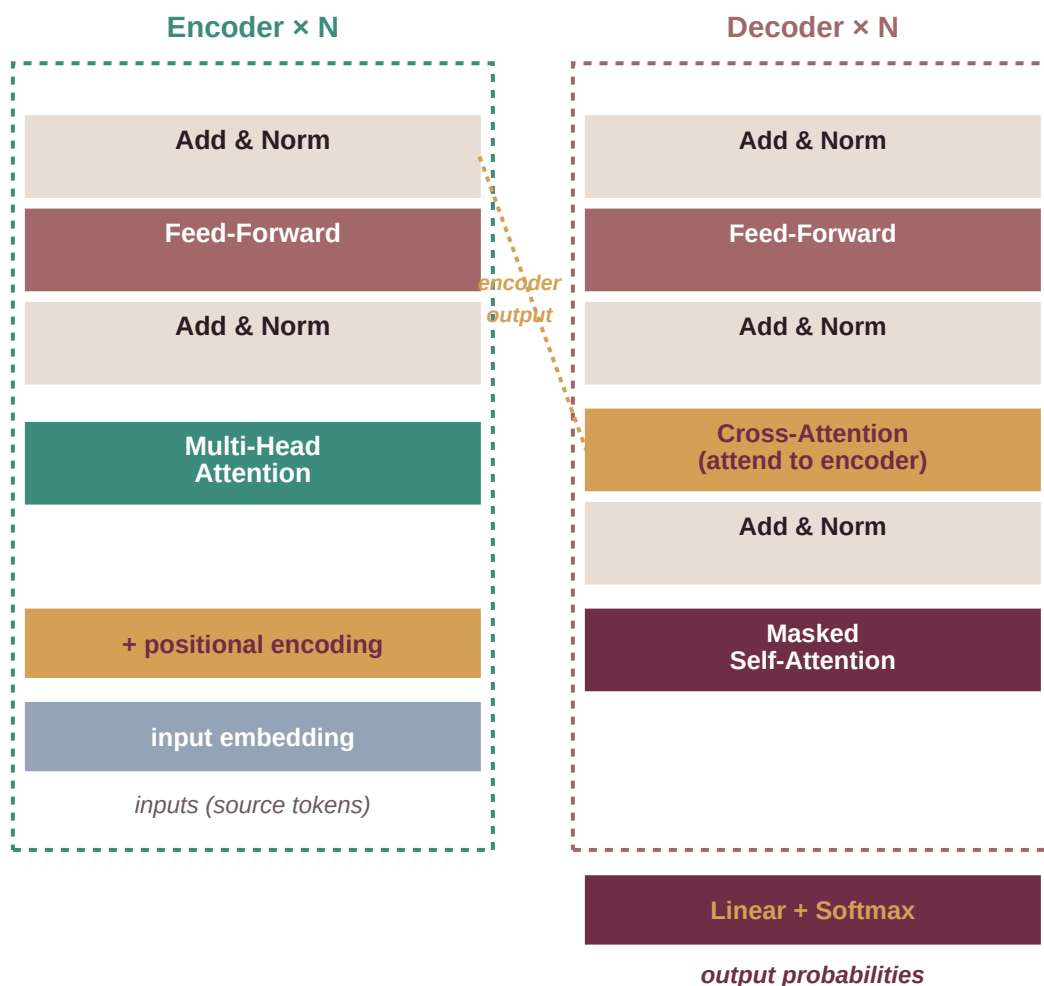
#### THE 'SELF' IN SELF-ATTENTION

Earlier attention mechanisms (Bahdanau-style) had the decoder attend to the encoder — two different sequences. **Self**-attention is when a sequence attends to **itself**. Every position attends to every other position in the same sequence. This is what allows the model to build rich contextual representations — each word's representation is computed by looking at all the other words in the same sentence.

## 4. Architecture: The Big Picture

The original Transformer was an **encoder-decoder** model designed for machine translation. The encoder reads the source sentence (e.g., English) and produces a sequence of contextualized representations. The decoder generates the target sentence (e.g., German) one token at a time, using both the encoder's output and the tokens it has already generated.

The Transformer architecture (Vaswani et al., 2017)



Encoder reads the source sentence; decoder generates the target sentence one token at a time.

**Modern LLMs (GPT, Llama) drop the encoder entirely and use only the decoder side.**

Figure 4.1 — The full Transformer architecture. The encoder (left) and decoder (right) are each stacks of  $N$  identical layers ( $N=6$  in the original paper). The decoder additionally has cross-attention layers that attend to the encoder's output.

### 4.1 The encoder

The encoder is a stack of  $N$  identical layers. In the original paper,  $N=6$ . Each layer has two sub-layers:

1. **Multi-head self-attention.** Each position attends to all positions in the same sequence (we'll explain multi-head in Section 5).
2. **Feed-forward network (FFN).** A small two-layer neural network applied to each position independently. This is where most of the model's parameters actually live.

Each sub-layer is wrapped with a residual connection and layer normalization (Section 6). The encoder transforms the input embeddings into a sequence of contextually-aware vectors that capture not just each word but its relationships to every other word.

## 4.2 The decoder

The decoder is also a stack of  $N$  identical layers. Each decoder layer has **three** sub-layers instead of two:

1. **Masked self-attention.** Like the encoder's self-attention, but with a causal mask: at position  $t$ , the decoder can only attend to positions  $\leq t$ . This prevents "cheating" during training (Section 6).
2. **Cross-attention (encoder-decoder attention).** The decoder attends to the encoder's output. Queries come from the decoder, keys and values from the encoder. This is how the decoder "reads" the source sentence.
3. **Feed-forward network.** Same as in the encoder.

## 4.3 The decoder-only variant — and why it's everything

Modern LLMs like GPT, Claude, and Llama don't use this full encoder-decoder architecture. They use **only the decoder side**, dropped of the cross-attention layer. The model becomes a single stack of "masked self-attention + FFN" layers, trained to predict the next token autoregressively. The encoder-decoder structure is mostly relegated to translation-specific models today.

This is worth internalizing: when you hear "transformer" in 2026, the speaker almost always means "decoder-only transformer." The 2017 architecture in the paper is the historical full version. The dominant production version is a simplification of it.

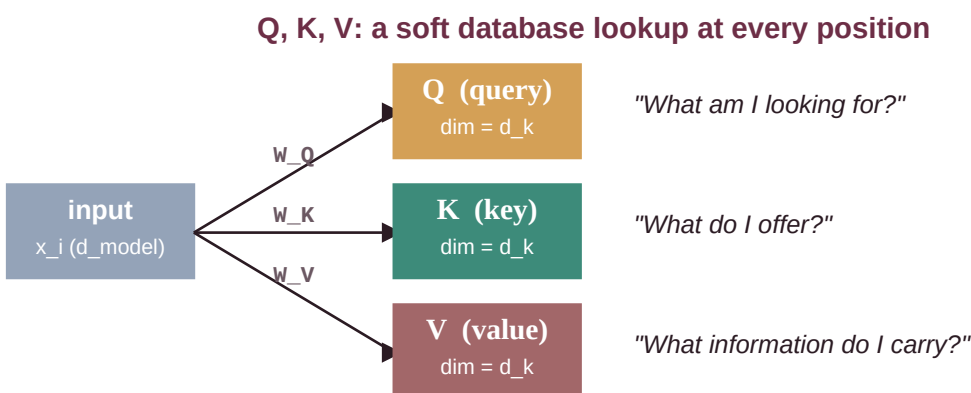
## 5. Attention in Depth

Now let's dig into how attention actually works. This is the most important section of the guide — internalize this and the rest follows.

### 5.1 Q, K, V — three views of the same input

For every input token, the Transformer computes three vectors: a **Query** (Q), a **Key** (K), and a **Value** (V). They're produced by passing the input embedding through three different learned weight matrices:

$$Q = x \cdot W_Q \quad K = x \cdot W_K \quad V = x \cdot W_V$$



Every token produces three vectors via three learned projection matrices.

**The whole point: attention will compare Queries to Keys, then pool Values.**

**Mental model: a soft, learned dictionary lookup at every layer.**

*Queries are the search terms. Keys are the index entries. Values are the content.*

Figure 5.1 — Each input embedding  $x$  is projected three times, producing a Query, a Key, and a Value vector. These three vectors play distinct roles in the attention computation.

The intuition: think of attention as a soft, learnable dictionary lookup. When you process a word and ask "what context do I need?", the answer comes in three parts:

**Query.** What you're searching for. "I'm processing the word 'It' — what should I focus on?"

**Key.** What's advertised at each position. Each word in the sentence says "here's what I am, in a form that can be matched against queries."

**Value.** The actual content delivered if a position is attended to. "If you decide I'm relevant, here's the information you'll get."

The Q and K are matched (via dot product) to compute attention scores. The scores are turned into weights via softmax. The weights are then applied to the V vectors to produce the output.

### 5.2 The scaled dot-product attention formula

The full formula, written out:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

This is the central equation. Every modern LLM runs some version of it billions of times per forward pass. Let's unpack it.

### Scaled dot-product attention, step by step

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

*the equation that runs every modern LLM*

- 1

**Compute similarity scores**  
 $Q \cdot K^T$

For each query, dot-product against every key. High score = relevant.
- 2

**Scale by  $\sqrt{d_k}$**   
 $(\cdot) / \sqrt{d_k}$

Prevents very large dot products from saturating softmax to 0 or 1.
- 3

**Softmax to get weights**  
 $\text{softmax}(\cdot)$

Convert scores to a probability distribution summing to 1.
- 4

**Weighted sum of values**  
 $(\cdot) \cdot V$

Pool the value vectors using the softmax weights. Output is one vector.

Shapes:  $Q, K, V$  are matrices of shape (sequence\_length,  $d_k$ ).

*The whole computation is two matrix multiplies plus a softmax — highly GPU-friendly.*

Figure 5.2 — The four steps of scaled dot-product attention. Two matrix multiplications surrounding a softmax. The whole operation is GPU-friendly and fully parallel across positions.

### Why the $\sqrt{d_k}$ scaling?

Dot products between random vectors of dimension  $d_k$  have variance proportional to  $d_k$ . If  $d_k$  is large (in the original paper,  $d_k = 64$ ), the raw dot products can become very large. Softmax of large values saturates — the largest value gets weight  $\sim 1.0$  and all others get  $\sim 0.0$ . Once you're in that regime, gradients for the non-maximum entries vanish and learning slows to a crawl.

Dividing by  $\sqrt{d_k}$  normalizes the variance back to roughly 1, keeping softmax in a regime where gradients flow well. This single scaling factor was one of the paper's important practical contributions.

## 5.3 Multi-head attention

Rather than computing attention once, the Transformer computes it **multiple times in parallel**, each time with different learned projection matrices. These parallel attention computations are called "heads." The original paper used 8 heads.

### Multi-head attention: 8 parallel views of the same input

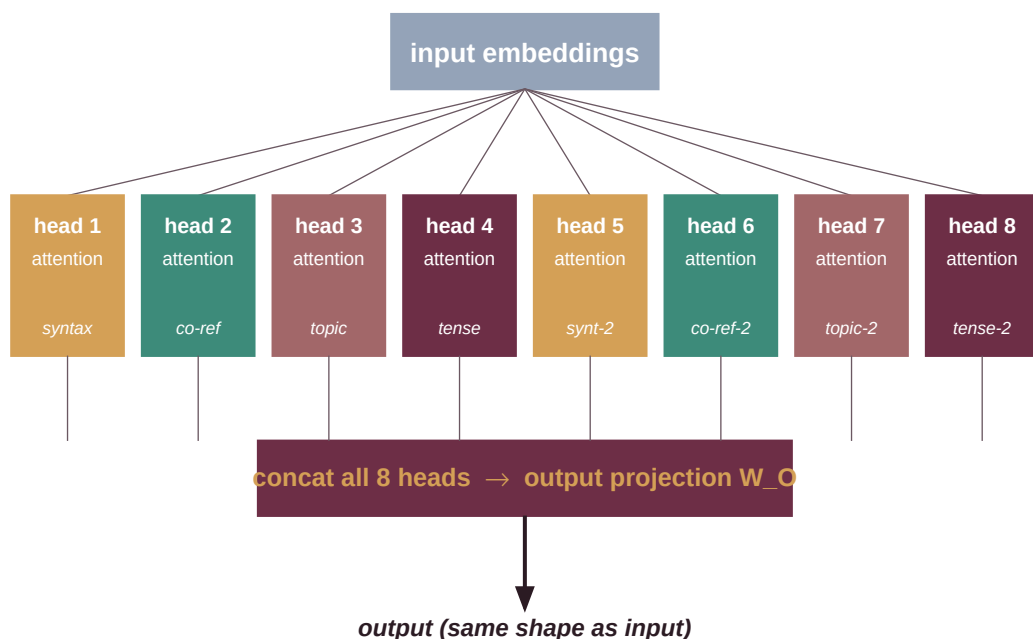


Figure 5.3 — Multi-head attention runs 8 separate attention computations in parallel, each with its own learned  $W_Q$ ,  $W_K$ ,  $W_V$  matrices. The outputs are concatenated and passed through a final output projection  $W_O$ .

Why have multiple heads? Different heads can learn to attend to different types of relationships. One head might learn to track syntactic dependencies (which noun goes with which verb), another might track coreference (which pronoun refers to which noun), another might track topic similarity, and so on.

Each head operates on a reduced dimension ( $d_{\text{model}} / \text{num\_heads}$ ). With  $d_{\text{model}} = 512$  and 8 heads, each head's  $Q/K/V$  vectors are 64-dimensional. After concatenating all 8 head outputs, you get back to 512 dimensions, which is then projected by  $W_O$  to produce the final output.

#### WHAT THE PAPER DIDN'T SAY (AND WHAT WE'VE LEARNED SINCE)

The paper claimed multi-head attention learns to specialize. Follow-up work (Voita et al. 2019, Michel et al. 2019) showed many heads are redundant — you can prune up to 60% of heads with minimal quality loss. The 'specialization' story is real but messier than the paper suggested. Modern variants like Multi-Query Attention (MQA) and Grouped-Query Attention (GQA) exploit this redundancy by sharing  $K$  and  $V$  across heads, saving memory at inference.

## 6. The Supporting Machinery

Attention is the headline feature, but the Transformer wouldn't work without several supporting pieces. This section covers the rest.

### 6.1 Positional encoding

Self-attention has a curious property: it's **permutation-equivariant**. If you shuffle the order of the input tokens, the outputs shuffle correspondingly. The attention mechanism itself doesn't know that "The cat sat" is different from "sat cat The" — it just sees a set of vectors.

For language, this is a problem. Word order matters enormously. So we need to inject positional information somehow.

The paper's solution: add a **positional encoding** vector to each input embedding before any attention is computed. The positional encoding for position  $pos$  is a fixed vector of sines and cosines at different frequencies:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_{model})}) \quad PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_{model})})$$

#### Positional encoding: how the model learns about word order

*Each token position gets a unique vector added to its embedding*



$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_{model})})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_{model})})$$



*Different dimensions oscillate at different frequencies — like a positional fingerprint.*

Figure 6.1 — Positional encoding uses sinusoidal patterns at different frequencies. Each position gets a unique fingerprint, and the model can compute relative positions easily because of trigonometric identities.

Why sines and cosines? Two reasons. First, each position gets a unique vector (no two positions look the same). Second, the model can easily learn to attend by *relative* position — because of the way sines and cosines combine,  $PE(pos + k)$  can be expressed as a linear function of  $PE(pos)$ , so the model can learn "attend  $k$  positions back" in a uniform way regardless of the absolute position.

Modern variants use different schemes — learned positional embeddings (BERT, GPT-2), rotary positional embeddings (RoPE, used in Llama and most modern models), or ALiBi (used in some Mistral variants). But the basic principle — inject position information additively into the embeddings — is the same.

## 6.2 Causal masking (for the decoder)

The decoder generates output tokens one at a time. At training time, we want to train it to predict each token from only the tokens that came before it. If we just used regular self-attention in the decoder, position  $t$  would be able to attend to positions  $t+1$ ,  $t+2$ , ... — effectively cheating by looking at the answer.

The fix: **causal masking**. Before applying softmax, set all attention scores for future positions to negative infinity. After softmax, these become zero. Position  $t$  can no longer see positions  $> t$ .

**Causal mask: decoder can only attend to past tokens**

| Query (row) | The | cat | sat | on | the | mat |
|-------------|-----|-----|-----|----|-----|-----|
| The         | ✓   | ×   | ×   | ×  | ×   | ×   |
| cat         | ✓   | ✓   | ×   | ×  | ×   | ×   |
| sat         | ✓   | ✓   | ✓   | ×  | ×   | ×   |
| on          | ✓   | ✓   | ✓   | ✓  | ×   | ×   |
| the         | ✓   | ✓   | ✓   | ✓  | ✓   | ×   |
| mat         | ✓   | ✓   | ✓   | ✓  | ✓   | ✓   |

allowed (current or past)

masked (future)

*Implementation: set masked scores to  $-\infty$  before softmax  $\rightarrow$  their weights become 0.*

**Why it matters: at training time, the decoder must predict word  $t$  without seeing words  $t+1$ ,  $t+2$ , ...**

*Figure 6.2 — The causal attention mask. Each row represents a query position; each column represents a key position. Only the lower-triangular part is allowed — every position attends only to itself and prior positions.*

All decoder-only LLMs (GPT, Claude, Llama) use causal masking. This is what makes them "autoregressive" — each token is predicted from the tokens before it.

## 6.3 Residual connections and layer normalization

Every sub-layer (each attention block and each FFN) is wrapped in two pieces of supporting machinery: a **residual connection** and **layer normalization**.

Every sub-layer is wrapped with a residual + LayerNorm

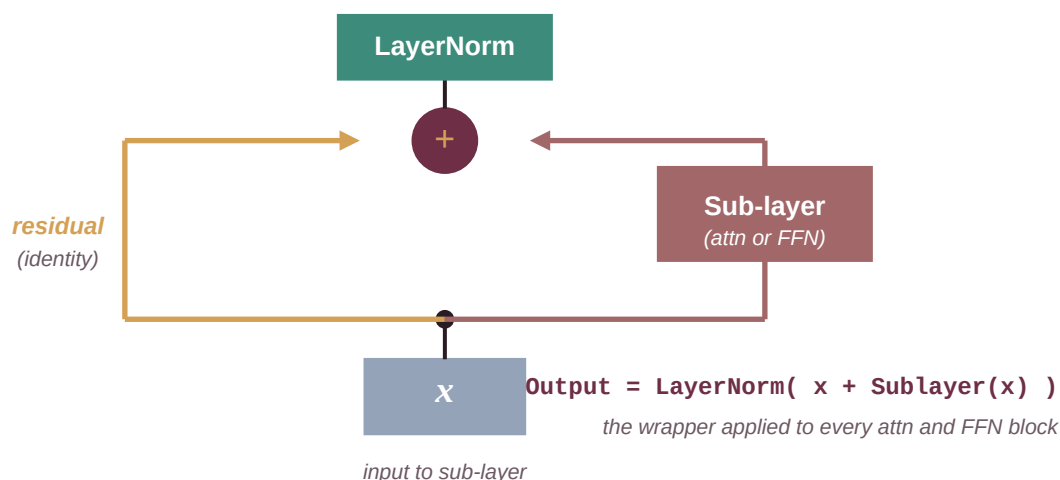


Figure 6.3 — Every sub-layer is wrapped as:  $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$ . The residual connection (skip path) is critical for training stability when stacking many layers.

### Residual connections

Instead of just applying the sub-layer and using its output, you *add* the sub-layer's output to its input. This means the sub-layer learns to produce a *correction* to its input rather than a brand-new representation. If the sub-layer learned to produce all zeros, the input would pass through unchanged.

Why it matters: with many stacked layers (12, 24, 96+ in modern models), gradients have to travel through many sub-layers during backprop. Residual connections create a direct "highway" for gradients to flow backward, preventing them from vanishing.

### Layer normalization

LayerNorm normalizes each token's hidden vector to have zero mean and unit variance, then applies a learned scale and shift. Unlike BatchNorm (which normalizes across the batch dimension), LayerNorm normalizes across the feature dimension of a single token. This keeps activations in a stable range as they pass through many layers.

Modern variants often place LayerNorm *before* the sub-layer rather than after (called "pre-norm"), which improves training stability further. The original paper used post-norm.

## 6.4 The feed-forward network (FFN)

After each attention block, every position passes through a two-layer feed-forward network, applied *identically and independently* to each position:

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

It's two linear layers with a ReLU in between. The intermediate dimension is typically 4× the model dimension — in the original paper,  $d_{\text{model}} = 512$  and the FFN intermediate dimension was 2048.

**The FFN is where most of the parameters actually live.** A 7-billion-parameter transformer has roughly 2/3 of its weights in the FFN layers, not in the attention layers. Attention does the mixing across positions; the FFN does the heavy computation at each position.

## 7. Code Walkthrough

Now let's see all of this as code. We'll build a minimal decoder-only transformer in PyTorch — about 80 lines for the full model. This is the kind of code you'd write to understand the architecture, not to train at scale.

### 7.1 Multi-head attention

The heart of the transformer:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, num_heads=8):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads          # per-head dim

        # The three projection matrices (W_Q, W_K, W_V) and the output W_O
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        B, T, D = x.shape  # batch, sequence length, d_model

        # 1. Project to Q, K, V – same input, three views
        Q = self.W_q(x)
        K = self.W_k(x)
        V = self.W_v(x)

        # 2. Split into heads: (B, T, D) -> (B, num_heads, T, d_k)
        Q = Q.view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        K = K.view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        V = V.view(B, T, self.num_heads, self.d_k).transpose(1, 2)

        # 3. Scaled dot-product attention
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)  # (B, h, T, T)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = F.softmax(scores, dim=-1)  # (B, h, T, T)
        out = attn @ V  # (B, h, T, d_k)

        # 4. Concat heads and project: (B, h, T, d_k) -> (B, T, D)
        out = out.transpose(1, 2).contiguous().view(B, T, D)
        return self.W_o(out)
```

The four numbered steps map directly to Figure 5.2. The reshape operations (view + transpose) are mechanical — they split the `d_model` dimension into `num_heads` parallel computations, which `Q @ K.transpose` handles in one fused matrix multiply.

## 7.2 A transformer block (one layer)

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, num_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model),
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Pre-norm variant (modern default)
        x = x + self.dropout(self.attn(self.norm1(x), mask=mask))
        x = x + self.dropout(self.ffn(self.norm2(x)))
        return x
```

Each block is: norm → attention → residual → norm → FFN → residual. The pre-norm placement (normalize *before* the sub-layer) is what most modern models use.

## 7.3 The full GPT-style model

```
class GPTLike(nn.Module):
    def __init__(self, vocab_size, d_model=512, num_heads=8,
                 num_layers=6, d_ff=2048, max_seq_len=1024):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_seq_len, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, num_heads, d_ff)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size)

    def forward(self, idx):
        B, T = idx.shape
        # Causal mask: lower triangular (allow past, mask future)
        mask = torch.tril(torch.ones(T, T, device=idx.device)).view(1, 1, T, T)

        pos = torch.arange(T, device=idx.device).unsqueeze(0)
        x = self.token_emb(idx) + self.pos_emb(pos)

        for block in self.blocks:
            x = block(x, mask=mask)

        x = self.norm(x)
        return self.head(x) # (B, T, vocab_size) - next-token logits per position
```

That's a complete GPT-style transformer. It's about 80 lines and runs the same fundamental computation as GPT-4 — the only differences are scale ( $d_{\text{model}} = 12,288+$  instead of 512;  $\text{num\_layers} = 96+$  instead of 6), better positional encoding (RoPE instead of learned absolute), and various efficiency tricks (FlashAttention, grouped-query attention). The mathematical core is exactly what's above.

### FROM 80 LINES TO PRODUCTION

Real production transformers have many additional concerns: FlashAttention for memory-efficient attention, KV caching for fast inference, mixed precision (bf16/fp16) for memory savings, gradient checkpointing for training memory, tensor parallelism for multi-GPU. None of this changes the math — it's all about making the same computation faster and more memory-efficient.

## 8. Limitations & What Came Next

The Transformer is foundational, but it has real limitations. Most of the interesting follow-up work since 2017 has been about addressing them.

### 8.1 Quadratic complexity in sequence length

The attention operation computes a similarity score between every pair of positions. For a sequence of length  $N$ , this is  $N \times N = N^2$  operations. Doubling the sequence length quadruples the compute and memory needed for attention.

For short sequences (a few hundred tokens), this is fine. For documents, codebases, or hours of conversation history, quadratic cost becomes prohibitive. A 100k-token context window requires storing a  $100k \times 100k$  attention matrix — 10 billion entries.

#### *Fixes that emerged*

**FlashAttention (Dao et al., 2022).** Doesn't change the math, just the implementation. Recomputes attention in tiles that fit in GPU SRAM (fast cache) instead of materializing the full  $N^2$  matrix in slower HBM. 2-4× speedup and 10× memory savings. Used by every modern LLM.

**Sparse attention.** Restrict attention to local windows (e.g., 1024 tokens around each position) plus a few global tokens. Used in Longformer, BigBird, and some Mistral variants.

**Linear attention approximations.** Performer, Linformer, and others approximate the softmax attention with kernel methods or low-rank projections to bring complexity from  $O(N^2)$  to  $O(N)$ . Quality usually suffers somewhat.

### 8.2 No built-in notion of position

Self-attention is permutation-invariant — it has to be told about positions via positional encoding. The original sinusoidal encoding worked but didn't extrapolate well to sequences longer than seen during training.

**Rotary positional embeddings (RoPE).** Used in Llama, Mistral, Qwen, and most modern open-weight models. Encodes position by *rotating* the Q and K vectors based on position, which has nice theoretical properties and extrapolates better.

**ALiBi (Attention with Linear Biases).** Add a linear bias to attention scores based on the distance between query and key. Simple and extrapolates well.

### 8.3 Inefficient for long generation

At inference time, a decoder-only transformer generates tokens one by one. For each new token, it needs to attend to all prior tokens — which means recomputing K and V for every position every step, or caching them.

**KV cache.** Save the K and V vectors from previous steps. New token only needs to compute its own Q, K, V; the prior K and V come from cache. Reduces per-step cost from  $O(N^2)$  to  $O(N)$ .

**Multi-Query Attention (MQA) / Grouped-Query Attention (GQA).** Standard multi-head attention has separate K and V projections per head. MQA shares one K and V across all heads; GQA shares across groups

of heads. Dramatically reduces the size of the KV cache at moderate quality cost. Used in Llama 2/3, Mistral, and most modern large models.

## 8.4 Hard to interpret

Attention weights were initially interpreted as "the model is looking at these words." Later research showed that attention weights don't always correspond to feature importance — the model can route information through low-attention paths via residual connections. The relationship between attention patterns and model behavior is more subtle than it appears.

Mechanistic interpretability research (Anthropic, others) studies what individual neurons, attention heads, and circuits actually compute. Progress has been made but transformers remain largely opaque compared to classical methods.

## 8.5 What came next architecturally

Since 2017, the Transformer architecture has been remarkably stable. Almost every modern LLM is recognizably the same architecture. The changes have been:

**Pre-norm** instead of post-norm (better training stability).

**RoPE or ALiBi** instead of sinusoidal positional encoding.

**SwiGLU** instead of ReLU in the FFN (slightly better quality).

**GQA** instead of full multi-head attention (KV cache savings).

**Mixture of Experts** (MoE): replace dense FFNs with sparse expert networks. Used in Mixtral, DeepSeek-V3, GPT-4 (reportedly).

**FlashAttention** at the implementation level for memory-efficient attention.

But the basic structure — stacked self-attention + FFN blocks with residual connections — hasn't changed. The 2017 paper got the architecture fundamentally right.

## 9. Adoption: The Transformer's Conquest

The Transformer is the most widely-adopted neural network architecture in history. Almost every modern AI system uses it. Here's a tour.

### 9.1 Language models

Every major language model is a Transformer:

**GPT family** (OpenAI): GPT-1 (117M), GPT-2 (1.5B), GPT-3 (175B), GPT-4, GPT-4o, GPT-5. All decoder-only Transformers.

**Claude family** (Anthropic): all decoder-only Transformers with various scale and training innovations.

**Gemini family** (Google): decoder-only Transformers with MoE variants.

**Llama family** (Meta): Llama 1, 2, 3, 4 — all decoder-only Transformers, the backbone of the open-source ecosystem.

**Mistral, DeepSeek, Qwen, Gemma**: all decoder-only Transformers, often with MoE for the larger variants.

**BERT family**: encoder-only Transformers, still widely deployed for classification, embeddings, and search ranking. Less glamorous than generative models but they run quietly in countless production systems.

### 9.2 Vision

After dominating NLP, Transformers came for computer vision. The Vision Transformer (ViT, Dosovitskiy et al. 2020) splits an image into patches, treats each patch like a word, and runs a standard Transformer on the sequence. It matched or beat CNNs at large scale.

Today, ViT variants power image classification, object detection, segmentation, and the image encoders inside every vision-language model (GPT-4V, Claude with vision, Gemini). CLIP, SigLIP, DINOv2 — all ViTs underneath.

### 9.3 Audio and speech

Whisper (OpenAI's speech recognition model) is an encoder-decoder Transformer. Wav2Vec, Conformer, and most modern speech systems use Transformer-based architectures. Music generation systems (Suno, MusicGen, Riffusion) are Transformer-based. Voice cloning systems (ElevenLabs, ChatTTS) use Transformers.

### 9.4 Biology and chemistry

AlphaFold-2 and AlphaFold-3 use Transformer-like attention mechanisms for protein structure prediction. The ESM family of protein language models uses standard Transformer architecture trained on protein sequences. RFdiffusion and related protein design tools use diffusion + Transformer combinations.

Molecule generation systems (for drug discovery), DNA sequence modeling (Enformer, Evo), and even single-cell biology (scGPT) all use Transformers.

## 9.5 Robotics and control

Vision-language-action models like RT-2 (Google DeepMind), PaLM-E, and OpenVLA use Transformer architectures that take in images + instructions and output robot actions. Decision Transformer reformulates reinforcement learning as a sequence prediction problem solvable by standard Transformers.

## 9.6 Why this universality?

Why does the same architecture work for text, images, audio, protein sequences, and robot actions? The answer is the attention mechanism's generality. Self-attention doesn't make strong assumptions about the structure of its input — it just learns to relate elements of a sequence to each other. Whatever the domain, if you can tokenize the inputs (split them into discrete pieces), you can run a Transformer on them.

Combined with scale and unlimited training data via self-supervised learning, this generality has made the Transformer the closest thing AI has to a universal architecture. Whether this lasts forever or whether something better comes along is an open question — but it's been seven years now and no credible challenger has emerged.

# 10. Worked Examples

Three concrete scenarios showing how to think about transformers in practice.

## 10.1 Example: Computing the memory budget for a 7B model

### Scenario

You want to run Llama-3-8B on a single GPU for inference. How much memory does it need?

### Calculation

**Weights:** 8 billion parameters  $\times$  2 bytes (fp16/bf16) = 16 GB.

**KV cache** for a sequence of length T:

```
2 (K and V)  $\times$  num_layers  $\times$  num_kv_heads  $\times$  head_dim  $\times$  T  $\times$  2 bytes = 2  $\times$  32  $\times$  8 (GQA)  $\times$  128  $\times$  T  $\times$  2  $\approx$  130 KB per token For T = 8192 tokens:  $\sim$ 1 GB
```

**Activation memory** during forward pass: a few hundred MB depending on implementation.

**Total:**  $\sim$ 18-20 GB for inference with an 8k context. Fits on a 24 GB consumer GPU (RTX 4090) but not on most data-center GPUs without quantization.

**With 4-bit quantization:** weights drop to  $\sim$ 4 GB, total  $\sim$ 6-7 GB. Fits on a 12 GB GPU comfortably. This is why QLoRA and 4-bit inference matter.

## 10.2 Example: Why your long-context call is so slow

### Scenario

You're building a RAG application. A short user query (50 tokens) returns quickly. A long-document analysis (50,000 tokens) takes 10 $\times$  longer per output token. Why?

### Explanation

Attention is  $O(N^2)$  in sequence length. The prefill phase (processing all prompt tokens) for a 50k-token prompt does  $\sim 50,000^2 = 2.5$  billion attention operations per layer. For a 32-layer model, that's 80 billion attention ops before you even start generating.

Once you start generating, each new token's attention is  $O(N)$  thanks to the KV cache — but with  $N=50k$ , that's still 50,000 operations per layer per token. With 32 layers, 1.6 million ops per output token.

The fix: don't put everything in context if you don't need to. Retrieval-augmented generation (RAG) explicitly selects only the most relevant chunks. Caching the KV state of a static prefix (system prompt, instructions) helps amortize the prefill cost across many queries.

## 10.3 Example: Picking `d_model` and `num_heads`

### Scenario

You're training a small transformer from scratch for a research project. What should `d_model` and `num_heads` be?

### *Guidance*

**d\_model** determines model capacity. Common values: 128, 256, 512 (small research), 1024 (small production), 4096 (Llama-7B), 8192 (Llama-70B), 12288 (GPT-3 175B).

**num\_heads** should divide `d_model` evenly. Common rule: `d_k` (per-head dimension) = 64 or 128. With `d_model` = 512 and `d_k` = 64, use `num_heads` = 8.

**num\_layers** typically 6-12 for small models, 32 for 7B, 80+ for 70B+.

**d\_ff** typically  $4 \times d_{\text{model}}$ . Modern variants using SwiGLU use  $\sim 8/3 \times d_{\text{model}}$ .

These ratios are empirical. The Chinchilla scaling laws (Hoffmann et al. 2022) give specific guidance on optimal model size given a training compute budget. For most projects, start with values close to a known working configuration (e.g., NanoGPT) and adjust from there.

# 11. Likely Interview Questions

If you understand the previous chapters, you can answer all of these.

**Q:** Walk me through self-attention.

**A:** For each token in the input, the model computes three vectors via three learned projections: a Query (Q), a Key (K), and a Value (V). To compute the new representation for position  $i$ , take  $Q_i$  and dot-product it against every  $K_j$  to get raw attention scores. Divide by  $\sqrt{d_k}$  (where  $d_k$  is the head dimension) to prevent softmax saturation. Apply softmax across the  $j$  dimension to get attention weights that sum to 1. Use those weights to take a weighted sum of the  $V_j$  vectors. The result is the new representation for position  $i$ , which now contains contextualized information from across the entire sequence.

---

**Q:** What does the  $\sqrt{d_k}$  scaling do, and why is it needed?

**A:** Dot products between random vectors of dimension  $d_k$  have variance proportional to  $d_k$ . For typical  $d_k$  values (64 or 128), raw dot products can become very large in magnitude. When softmax is applied to large values, it saturates — the largest score gets weight  $\sim 1$  and others get  $\sim 0$ . Once saturated, gradients for the non-maximum entries become essentially zero, and learning stalls. Dividing by  $\sqrt{d_k}$  normalizes the variance back to  $\sim 1$ , keeping softmax in a healthy gradient regime.

---

**Q:** Why is multi-head attention better than single-head?

**A:** Different heads can specialize in different relationships — one might track syntactic dependencies, another coreference, another topic similarity. Each head operates in a lower-dimensional subspace ( $d_{\text{model}} / \text{num\_heads}$ ), and the concatenated output is projected through  $W_O$  to combine the heads' findings. Empirically, multi-head consistently beats single-head at the same parameter budget. Follow-up research has shown many heads are redundant — which is why modern models use Grouped-Query Attention to share K/V across heads while keeping separate Q projections.

---

**Q:** Why does the decoder need a causal mask?

**A:** The decoder is trained to predict the next token given the previous tokens. If position  $t$  could attend to positions  $t+1$ ,  $t+2$ , ..., it would have access to the answer during training — trivially predicting each token by looking it up. The causal mask sets all attention scores for future positions to  $-\infty$  before softmax, making their weights zero. This forces the model to predict each token using only information from the past, which is the same constraint it faces at inference time.

---

**Q:** What is positional encoding and why is it needed?

**A:** Self-attention is permutation-equivariant — it produces the same outputs regardless of the input order. For natural language, where word order matters, this is a problem. Positional encoding adds position information to each token embedding before any attention is computed. The original paper used fixed sinusoidal encodings with different frequencies. Modern models use learned absolute embeddings (BERT, GPT-2), Rotary Position Embeddings (Llama, Mistral), or ALiBi linear biases — all serving the same purpose.

---

**Q:** What's the difference between encoder-only, decoder-only, and encoder-decoder transformers?

**A:** Encoder-only (like BERT) processes a sequence bidirectionally — every position can attend to every other position, including future ones. Used for understanding tasks: classification, embeddings, named entity recognition. Decoder-only (like GPT, Llama, Claude) uses causal masking so each position only attends to past positions. Trained for next-token prediction. Used for generation. Encoder-decoder (like the original Transformer, T5) has both — the encoder processes the input bidirectionally and the decoder attends to the encoder's output via cross-attention while generating. Used for sequence-to-sequence tasks like translation. Modern LLMs are overwhelmingly decoder-only.

---

**Q:** Why is attention  $O(N^2)$ ? What does that mean in practice?

**A:** Attention computes a similarity score between every pair of positions. For a sequence of length  $N$ , this is  $N \times N = N^2$  operations and an  $N \times N$  attention matrix. Doubling sequence length quadruples both compute and memory. Practically: a 1k-token context is cheap, an 8k-token context is manageable, a 100k-token context is expensive, and a 1M-token context is extremely expensive. Fixes include FlashAttention (better memory access patterns), sparse attention (only attend within windows), and linear attention approximations (Performer, Linformer).

---

**Q:** What is a KV cache and why does it matter?

**A:** During autoregressive generation, each new token must attend to all prior tokens. Naively, you'd recompute  $Q$ ,  $K$ ,  $V$  for every position at every step. The KV cache stores  $K$  and  $V$  from previous steps so you only compute  $Q$ ,  $K$ ,  $V$  for the new token. This reduces per-token generation cost from  $O(N^2)$  to  $O(N)$ . The KV cache size scales with batch size, sequence length, `num_layers`, `num_heads`, and `head_dim` — it can easily exceed model weight memory for long sequences. Multi-Query Attention and Grouped-Query Attention reduce KV cache size by sharing  $K$  and  $V$  across attention heads.

---

**Q:** Where do most of the parameters in a transformer actually live?

**A:** Most people guess attention, but the FFN layers contain  $\sim 2/3$  of the parameters. For each transformer block, attention has  $4 \times d_{\text{model}}^2$  parameters ( $W_Q$ ,  $W_K$ ,  $W_V$ ,  $W_O$ ) while the FFN has  $2 \times d_{\text{model}} \times d_{\text{ff}}$  parameters — with  $d_{\text{ff}} = 4 \times d_{\text{model}}$ , that's  $8 \times d_{\text{model}}^2$  for the FFN versus  $4 \times d_{\text{model}}^2$  for attention. This is why Mixture of Experts works: replace the dense FFN with sparse expert networks and you can dramatically scale the parameter count without increasing inference cost.

---

**Q:** If you had to explain why the Transformer beat RNNs, what would you say?

**A:** Three things working together. (1) Parallelism: RNNs process tokens sequentially, so doubling the sequence length doubles the wall-clock time even on infinite hardware. Transformers process all tokens at once, exploiting GPU parallelism fully. (2) Direct connections across distance: RNNs had to compress information from earlier in the sequence into a single hidden vector, losing detail over long distances. In transformers, any two positions are one attention operation apart — no information bottleneck. (3) Better gradient flow: residual connections plus shorter computational paths mean gradients propagate cleanly even through deep stacks of layers. Combined, these enabled training larger models on larger datasets, which is what unlocked the scaling era.

# Appendix A. Key Equations & Terms

## Equations

### Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

The central operation. Compare every query to every key, scale, softmax, then pool values.

### Q/K/V projections

$$Q = x \cdot W_Q, K = x \cdot W_K, V = x \cdot W_V$$

Three learned projections of the same input.

### Multi-head attention

$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$$

$h$  parallel attention operations, concatenated and projected.

### Per-head dimension

$$d_k = d_{\text{model}} / \text{num\_heads}$$

Each head operates in a reduced-dimensional subspace.

### Feed-forward network

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

Two linear layers with ReLU; intermediate dim typically  $4 \times d_{\text{model}}$ .

### Sub-layer with residual + norm

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Wraps every attention and FFN block. Pre-norm variant:  $\text{norm}(x)$  inside Sublayer.

### Positional encoding (sinusoidal)

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}), \text{PE}(\text{pos}, 2i+1) = \cos(\dots)$$

Fixed sine/cosine encoding from the original paper. Modern variants use RoPE or ALiBi.

## Terms

### Self-attention

Attention applied to a single sequence — each position attends to every other position in the same sequence. Contrast with cross-attention, where queries come from one sequence and keys/values from another.

### Query, Key, Value (Q, K, V)

Three vectors produced per token by linear projections. Queries are matched to keys to compute attention weights; values are then pooled by those weights.

|                            |           |                                                                                                                                                                                                                                                              |
|----------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Head</b>                |           | One parallel attention computation. Multi-head attention runs $h$ heads in parallel with different learned projections, then concatenates their outputs.                                                                                                     |
| <b>d_model</b>             |           | The model's hidden dimension. The size of each token's representation vector throughout the network. Common values: 512 (small), 4096 (Llama-7B), 12288 (GPT-3).                                                                                             |
| <b>d_k</b>                 |           | Per-head dimension. Typically $d_{\text{model}} / \text{num\_heads}$ . Most modern models use $d_k = 64$ or $128$ .                                                                                                                                          |
| <b>Causal mask</b>         |           | An attention mask that prevents each position from attending to future positions. Used in decoder-only models and the decoder of encoder-decoder models. Implemented by setting future scores to $-\infty$ before softmax.                                   |
| <b>Positional encoding</b> |           | Information added to token embeddings so the model knows about word order. Original paper used fixed sinusoidal patterns. Modern variants include RoPE (rotary), ALiBi, and learned absolute embeddings.                                                     |
| <b>KV cache</b>            |           | Cached K and V vectors from previous tokens, used during autoregressive generation to avoid recomputing them. Critical for efficient inference.                                                                                                              |
| <b>FlashAttention</b>      |           | An attention implementation that avoids materializing the full $N \times N$ attention matrix by computing it in tiles that fit in GPU SRAM. 2-4× faster and uses 10× less memory than naive attention.                                                       |
| <b>Encoder-only</b>        | <b>vs</b> | Three model topologies. Encoder-only (BERT) processes inputs bidirectionally for understanding tasks. Decoder-only (GPT, Llama) uses causal masking for generation. Encoder-decoder (T5, the original Transformer) combines both for translation-like tasks. |
| <b>decoder-only</b>        | <b>vs</b> |                                                                                                                                                                                                                                                              |
| <b>encoder-decoder</b>     |           |                                                                                                                                                                                                                                                              |
| <b>Pre-norm</b>            | <b>vs</b> | Whether LayerNorm is applied before or after each sub-layer. The original paper used post-norm. Pre-norm is more stable for very deep models and is the modern default.                                                                                      |
| <b>post-norm</b>           |           |                                                                                                                                                                                                                                                              |

## Further reading

**Original paper:** Vaswani et al., "Attention Is All You Need" (2017). arXiv:1706.03762. Short, dense, foundational. Required reading.

**The Illustrated Transformer** (Jay Alammar, 2018). Visual blog post that made transformers click for a generation of practitioners. Search the title.

**NanoGPT** (Andrej Karpathy). A minimal, readable implementation of a GPT-style decoder-only transformer. ~300 lines including training. Highly recommended.

**FlashAttention paper** (Dao et al., 2022). The most impactful follow-up from an implementation perspective. arXiv:2205.14135.

**RoFormer / RoPE paper** (Su et al., 2021). The positional embedding scheme used in most modern LLMs. arXiv:2104.09864.

**Companion study guides:** see the LoRA guide for parameter-efficient fine-tuning, and the LLM Evolution companion for how the Transformer fits into the broader history of language modeling.